

## TP 16 — Le tri par sélection au banc d'essai

Semaine 16 ■ Fichiers dans NSI/TP16/ ■ 2 h

### Objectifs

- Implémenter le tri par sélection et le valider sérieusement
- Vérifier expérimentalement le coût en  $n^2/2$
- Visualiser le tri pas à pas

### Partie A — Implémentation validée — tri.py

(40 min)

1. Implémenter `tri_selection(t)` (cours), docstring « MODIFIE t » comprise.
2. Jeu de tests : cas normal, déjà trié, ordre inverse, doublons, vide, un élément.
3. Le test ultime (exercice 6) — vérifier les **deux** propriétés sur 100 tableaux aléatoires :

```
import random

def test_masse():
    for _ in range(100):
        t = [random.randint(0, 50) for _ in range(20)]
        attendu = sorted(t)          # l'arbitre officiel
        tri_selection(t)
        assert t == attendu
    print("test_masse : OK")
```

(On s'autorise `sorted` **uniquement** comme arbitre de test — jamais dans le tri!)

4. ★ Casser le tri (remplacer `range(i + 1, n)` par `range(i, n)` puis par `range(i + 1, n - 1)`) : quels tests détectent quoi ?

### Partie B — Le compteur de comparaisons — banc.py

(35 min)

1. `tri_selection_compte(t)` — copie du tri qui **renvoie** le nombre de comparaisons effectuées.
2. Vérifier sur  $n = 10$  : exactement  $45 = 10 \times 9/2$ .
3. Afficher le tableau :  $n$ , comparaisons mesurées,  $n(n-1)/2$  — pour  $n = 10, 100, 1000$ .
4. Trier un tableau **déjà trié** de 1000 éléments : combien de comparaisons ? Confirmer le cours.

### Partie C — Chronométrage — même fichier

(25 min)

1. Chronométrer (`time.perf_counter`) le tri de tableaux aléatoires de 1000, 2000, 4000, 8000 éléments.
2. À chaque doublement, par combien la durée est-elle multipliée ? Est-ce cohérent avec  $O(n^2)$  ?
3. Extrapoler : durée estimée pour un million d'éléments ? (Ne pas lancer ! Calculer.)

### Partie D — Le tri qui s'affiche ★★ — visu.py

(20 min)

Faire afficher le tableau après chaque échange, avec la zone triée entre crochets :

```
[1] 3 8 5  
[1 3] 8 5  
[1 3 5] 8
```

Indice : dans la boucle externe, construire la chaîne avec deux tranches `t[:i+1]` et `t[i+1:]`. Tester sur un tableau mélangé par `random.shuffle(t)`.